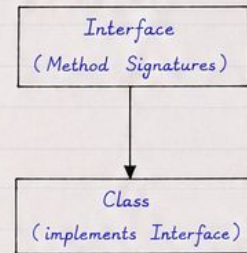


1. Abstract Class and Interface (एबस्ट्रेक्ट क्लास और इंटरफेस)

(A) Defining an Interface (इंटरफेस की परिभाषा) :

- Interface एक reference type होता है जो केवल method signatures को define करता है।
- Interface में variables default रूप से public, static और final होते हैं।
- Interface के methods default रूप से public और abstract होते हैं (Java 8 के बाद default और static methods भी हो सकते हैं)।
- Interface को implements keyword के द्वारा class में implement किया जाता है।
- Interface multiple inheritance की सुविधा देता है।

Concept Diagram :



Syntax :

```
interface InterfaceName {  
    // constant (public static final)  
    int MAX = 100;  
  
    // abstract method  
    void show();  
    void display(int x);  
  
    // default method (optional)  
    default void msg() {  
        System.out.println("Default Method");  
    }  
  
    // static method (optional)  
    static void info() {  
        System.out.println("Static Method");  
    }  
}
```

Example :

```
interface Drawable {  
    void draw();  
    void color();  
    default void show() {  
        System.out.println("Default  
Method");  
    }  
}
```

```
class Circle implements Drawable {  
    public void draw() {  
        System.out.println("Drawing  
Circle");  
    }  
    public void color() {  
        System.out.println("Color is  
Red");  
    }  
}
```

Test Class

```
class Test {  
    public static void main(String[] args) {  
        Circle c = new Circle();  
        c.draw(); // Drawing Circle  
        c.color(); // Color is Red  
        c.show(); // Default Method  
        Drawable.info(); // Static Method  
    }  
}
```

→ Circle class
Drawable interface
को implement कर
रही है और उसके
methods को define
कर रही है।

(B) Difference Between Class and Interface (क्लास और इंटरफेस में अंतर) :

Sr. No.	Class (क्लास)	Interface (इंटरफेस)
1.	Class एक blueprint हैं जिससे objects बनाए जाते हैं।	Interface एक contract हैं जिसे class implement करती हैं।
2.	Class में fields, methods, constructors सभी हो सकते हैं।	Interface में केवल method signatures होते हैं (fields केवल constants होते हैं)।
3.	Class के methods के पास body हो सकती हैं।	Interface के methods default रूप से abstract होते हैं।
4.	Class एक ही superclass को extend कर सकती हैं।	Interface multiple interfaces को extend कर सकती हैं।
5.	Class में access modifiers का use किया जा सकता है (private, protected, public, default)।	Interface के methods default रूप से public होते हैं।
6.	Class में constructor हो सकते हैं।	Interface में constructors नहीं होते हैं।
7.	Class में instance variables हो सकते हैं।	Interface में variables केवल public static final (constants) होते हैं।
8.	Class concrete और abstract दोनों हो सकती हैं।	Interface केवल abstract type होता है (Java 8 के बाद default methods संभव)।
9.	Class को extends keyword से inherit किया जाता है।	Interface को implements keyword से implement किया जाता है।
10.	Class state और behavior दोनों define करती हैं।	Interface केवल behavior (methods) define करता है।
11.	Class single inheritance support करता है।	Interface multiple inheritance support करता है।

Important Points :

- ✓ Interface 100% abstraction को support करता है।
- ✓ Interface loose coupling और flexible design के लिए useful होता है।
- ✓ Real-world में Java Collection Framework mostly interfaces पर based है (List, Set, Map आदि)।

1. Abstract Class and Interface (एक्स्ट्रेक्ट क्लास और इंटरफेस).

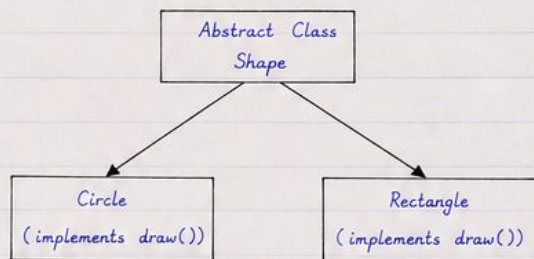
(A) Key Points of Abstract Class (एक्स्ट्रेक्ट क्लास के मुख्य बिंदु):

1. Abstract class को abstract keyword के द्वारा declare किया जाता है।
2. Abstract class को instantiate नहीं किया जा सकता (object नहीं बनाता)।
3. Abstract class में abstract (incomplete) methods और concrete (complete) methods दोनों हो सकते हैं।
4. Abstract methods का body नहीं होता, केवल declaration होता है।
5. Abstract class में constructors, instance variables, static members सभी हो सकते हैं।
6. Abstract class को extends करने वाली subclass को उसके सभी abstract methods को implement करना होता है, otherwise subclass भी abstract बनानी पड़ती है।
7. Abstract class का use code reusability और common behavior define करने के लिए होता है।
8. A class can extend only one abstract class (single inheritance).
9. Abstract class का reference लेकर subclass का object बनाया जा सकता है।

Example :

```
abstract class Shape {  
    abstract void draw();           // abstract method  
    void color() {                  // concrete method  
        System.out.println("Color is Red");  
    }  
}  
  
class Circle extends Shape {  
    void draw() {  
        System.out.println("Drawing Circle");  
    }  
}
```

Diagram :



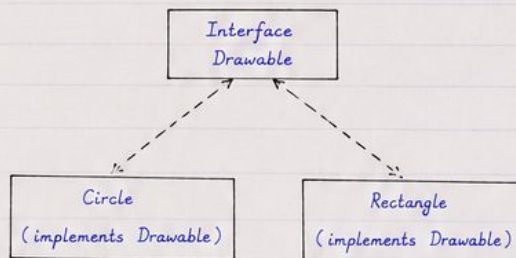
(B) Key Points of Interface (इंटरफेस के मुख्य बिंदु):

1. Interface को interface keyword के द्वारा declare किया जाता है।
2. Interface को instantiate नहीं किया जा सकता।
3. Interface में केवल abstract methods होते हैं (Java 8 के पहले)।
Java 8 के बाद, default methods और static methods भी हो सकते हैं।
4. Interface के सभी methods public और abstract होते हैं (by default)।
5. Interface के variables हमेशा public, static और final होते हैं (constants)।
6. Interface को implements keyword के द्वारा class implement करती है।
7. A class एक या अधिक interfaces को implement कर सकती है।
8. Interface multiple inheritance को support करता है।
9. Interface का use जब किसी behavior (capability) को define करना हो, तब किया जाता है।

Example :

```
interface Drawable {  
    void draw();           // abstract method  
    default void show() {   // default method (Java 8+)  
        System.out.println("Default Show Method");  
    }  
}  
  
class Circle implements Drawable {  
    public void draw() {  
        System.out.println("Drawing Circle");  
    }  
}
```

Diagram :



Quick Difference (Abstract Class vs Interface):

Feature	Abstract Class	Interface
1. Keyword	abstract class	Interface
2. Methods	Abstract + Concrete methods	Only Abstract methods (pre Java 8) (Java 8+ default, static methods)
3. Variables	Instance variables allowed	Only public static final (constants)
4. Constructors	Yes	No
5. Inheritance	Single inheritance	Multiple inheritance
6. Use	Common code + behavior	Define capability / behavior

1. Abstract Class and Interface (एक्स्ट्रेक्ट क्लास और इंटरफेस).

→ Abstract class और Interface दोनों ही abstraction को implement करने के लिए use होते हैं, लेकिन इनके design और use में कई differences होते हैं।

Difference Between an Abstract Class & Interface :

Sr. No.		Abstract Class	Interface
1.	Keyword	abstract keyword का use होता है।	interface keyword का use होता है।
2.	Purpose	Common behavior और state provide करने के लिए use होता है।	Capability define करने के लिए use होता है।
3.	Methods	Abstract (incomplete) और Concrete (complete) दोनों methods हो सकते हैं।	Default रूप से केवल Abstract methods होते हैं (Java 8 के पहले)। (Java 8 के बाद default और static methods भी हो सकते हैं।)
4.	Method Body	Abstract methods का body नहीं होता, Concrete methods का body होता है।	Default रूप से methods का body नहीं होता (Java 8 के पहले)।
5.	Variables	Instance variables और static variables दोनों हो सकते हैं।	केवल public static final (constants) होते हैं।
6.	Access Modifiers	private, protected, public सभी use हो सकते हैं।	Methods default रूप से public होते हैं। (private methods Java 9 से allowed)
7.	Constructors	Constructors होते हैं।	Constructors नहीं होते।
8.	Inheritance / Implementation	एक class केवल एक abstract class को extend कर सकती है।	एक class multiple interfaces को implement कर सकती है।
9.	Keyword for extend/implement	extends keyword का use होता है।	implements keyword का use होता है।
10.	State (Data)	State (instance variables) maintain कर सकते हैं।	State maintain नहीं कर सकते (केवल constants)।
11.	Performance	Interface की तुलना में थोड़ा कम optimized होता है।	Abstract class की तुलना में अधिक optimized होता है।
12.	Multiple Inheritance	Multiple inheritance support नहीं करता।	Multiple inheritance support करता है।
13.	When to use	जब closely related classes में common code हो, तब use करें।	जब unrelated classes में common capability हो, तब use करें।
14.	Backward Compatibility	New methods add करने पर subclass पर कोई effect नहीं पड़ता।	New methods add करने पर सभी implementing classes को implement करना पड़ता है।
15.	Example	Animal (abstract class)	Runnable, Serializable, Comparable

Abstract Class Example :

```
abstract class Shape {
    int sides; // instance variable

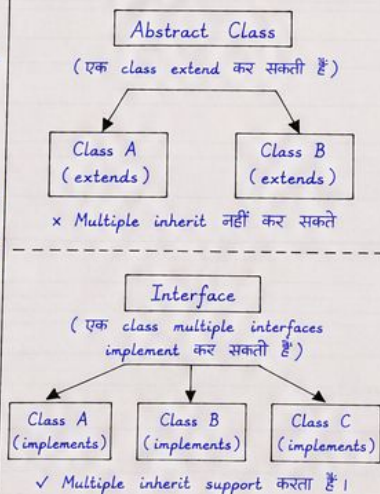
    Shape(int s) { // constructor
        sides = s;
    }

    abstract void area(); // abstract method

    void display() { // concrete method
        System.out.println("This is a shape");
    }
}

class Circle extends Shape {
    Circle() { super(0); }
    void area() {
        System.out.println("Area of Circle");
    }
}
```

Diagram :



Interface Example :

```
interface Drawable {
    void draw(); // abstract method
}

class Circle implements Drawable {
    public void draw() {
        System.out.println("Drawing Circle");
    }
}

class Rectangle implements Drawable {
    public void draw() {
        System.out.println("Drawing Rectangle");
    }
}
```

Important Points :

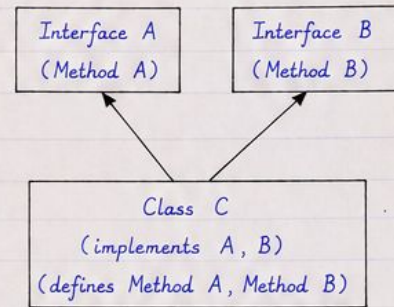
- ✓ Abstract class "is-a" relationship को represent करता है।
- ✓ Interface "can-do" या "ability" को represent करता है।
- ✓ Design के अनुसार सही choice करना important होता है।

1. Abstract Class and Interface (एबस्ट्रेक्ट क्लास और इंटरफेस)

Implementation of Multiple Inheritance through Interface :

- Java में class multiple inheritance को support नहीं करता, लेकिन interface की मदद से हम multiple inheritance achieve कर सकते हैं।
- एक class multiple interfaces को implement कर सकती है।
- इसमें class को सभी interfaces के abstract methods को implement करना होता है।
- इस प्रकार interface multiple inheritance का safe और effective तरीका प्रदान करता है।

Concept Diagram :



- Class C ने Interface A और Interface B दोनों को implement किया है।

Example :

```
interface Printable {
    void print();
}

interface Showable {
    void show();
}

class Document implements Printable, Showable {
    public void print() {
        System.out.println("Printing the document...");
    }

    public void show() {
        System.out.println("Showing the document...");
    }
}

class Test {
    public static void main(String[] args) {
        Document d = new Document();
        d.print();    // Printable interface का method
        d.show();     // Showable interface का method
    }
}
```

Output :

```
Printing the document...
Showing the document...
```

How it Works ?

- Document class ने Printable और Showable दोनों interfaces को implement किया है।
- इसलिए इसे दोनों interfaces के methods (print() और show()) को implement करना पड़ा।
- इस तरह एक class multiple interfaces को implement करके multiple inheritance का लाभ उठाती है।

Key Points / Advantages :

	Multiple Inheritance through Interface	Multiple Inheritance through Class
1.	एक class multiple interfaces को implement कर सकती है।	Class multiple classes को extend नहीं कर सकती।
2.	इसमें ambiguity की समस्या नहीं होती क्योंकि interface में केवल method signatures होते हैं।	Multiple class inheritance में ambiguity हो सकती है (दोनों parent class में same method होने पर)।
3.	Java 8 के बाद interface में default और static methods की सुविधा है।	Class inheritance rigid होता है और tight coupling बढ़ाता है।
4.	Interface 100% abstraction provide करता है।	Class partial abstraction provide करती है।
5.	Code reusability और flexibility अधिक होती है।	Code reusability limited होती है।

Important Note :

- ✓ Interface के द्वारा हम multiple inheritance को implement करते हैं, जो Java को powerful और flexible बनाता है।
- ✓ Best Practice : जब behavior define करना हो, तब interface का use करें।